

A scenic mountain landscape with green hills, rocky cliffs, and a cloudy sky. The foreground shows a rocky, grassy slope. In the middle ground, there are steep, rocky cliffs and green hills. The background features more distant, hazy mountain ranges under a sky filled with white and grey clouds.

AI & Software Engineering Workshop

Week 1, Day 5: Build Your Own Bot

Edik Simonian, Summer 2026

First: fix your git identity

Swapped seats? This machine may still be signed in as **yesterday's student** — so your pushes fail, or commits land under the wrong name.

```
brew install gh      # GitHub CLI – once per machine
gh auth logout       # drop the previous student's login
gh auth login        # GitHub.com → HTTPS → Login with a web browser
gh auth setup-git    # make git push as YOU
```

Then tell git who you are (run inside your bot folder):

```
git config user.name "Your Name"
git config user.email "the-email-on-your-github"
```

"Login with a web browser" works even if you sign in to GitHub with Google — it's still GitHub's OAuth.

Today: you're the engineer

Four days, four superpowers:

- **Day 1**: a bot that replies · **Day 2**: a personality
- **Day 3**: commands that think, and memory · **Day 4**: live on the internet

No new theory today. You **design and ship your own bot**, start to finish.

Same repo, same skills. The idea is yours.

Pick your idea

Choose one, or invent your own:

-  **Translator**: Armenian ↔ English on demand
-  **Quiz master**: asks, scores, keeps streaks (*you have a store!*)
-  **Mood reader**: guesses the mood of a message
-  **Summarizer**: paste a wall of text, get three lines

It must be **live on your deployed bot** by demo time.

Plan before you code

Before touching the keyboard, write **three lines**:

1. **The command(s)**: what does the user type? `/quiz`, `/translate <text> ...`
2. **Does it think?** A static reply, or does it call the AI (`ask_ai`)?
3. **Does it remember?** One-shot, or does it use the store (`note:{id}` pattern)?

A feature you can describe in three lines, you can build.

Build it: you lead, Claude Code assists

Same recipe as all week: **handler** → `/help` **entry** → **test** → **restart**.

- Ask Claude Code in plain English; it edits `bot/handlers.py` and can run the bot
- **Read every line.** Can't explain it? Ask it to explain, or undo it
- Add a test in `tests/`; make test **green** before you ship

You're the engineer. Claude Code is the assistant. You explain your code at demo.

Ship it live

You deployed yesterday, so today it's one step:

```
git commit -am "Add my feature" && git push
```

~3 seconds later your live bot has the feature. Push-to-deploy does the rest.

- No push-to-deploy set up? `make deploy-pa` still works
- Verify on your **live** bot, feature working, before demo

Demo day

1. Post your bot's username for the room
2. Try everyone's bot, find their feature
3. Vote: funniest persona, most useful feature, best surprise

Same bot you started on Day 1, now it's yours, and it lives on the internet.

Next week

Your bot's brain is someone else's model behind an API.

Week 2: we train our own. 63 GB of Armenian text, one GPU, a 350-million-parameter model, built from scratch, fine-tuned to chat, and plugged into *this* bot.